# Software Engineering Project Report

## (CSDC-0338)

Department of Computer Science and Engineering

Dr. B.R. Ambedkar National Institute of Technology Jalandhar,
Punjab-144008 (Jan-June 2026)

Submitted to:

Dr. Himanshu Verma
Department of Computer Science & Engineering

Submitted by:
Monika - 23117023
Sajal - 23117032

Team Number: 2

Project Title: Mess Management System

Group: DSE – G8

# Table of Contents


| S.No | Topic | Page |
|---|---|---|
| 1 | Project Overview | 1 |
| 2 | Problem Statement | 1 |
| 3 | Objectives | 2 |
| 4 | System Architecture | 3-4 |
| 5 | Database Design | 5-6 |
| 6 | Technology Stack | 7 |
| 7 | Features & Modules | 8 |
| 8 | Screenshots | 9-15 |
| 9 | Diagrams | 16-19 |
| 10 | API Documentation | 20-21 |
| 11 | Deployment | 22 |
| 12 | Future Scope | 23 |
| 13 | Conclusion | 23 |
| 14 | References | 23 |

# 1. Project Overview

SmartMess is a full-stack web application built on the MERN stack (MongoDB style architecture using Supabase PostgreSQL) that digitalizes hostel mess management. It provides a centralized platform for students and administrators to manage daily menus, track meal attendance, collect feedback ratings, and handle complaints efficiently.

| | |
|---|---|
| **Project Name** | Hostel Mess Management System |
| **Project Type** | Full-Stack Web Application (Academic ) |
| **Tech Stack** | MERN (React, Node.js, Express, Supabase PostgreSQL) |
| **Developer** | Sajal , Monika |
| **GitHub** | https://github.com/monikaasidhu/Mess-Management-System |
| **Backend API** | https://messmanagementsystem-4ngq.onrender.com |
| **Deployment** | Vercel (Frontend) + Render (Backend) |
| **Database** | Supabase (PostgreSQL) |

# 2. Problem Statement

Traditional hostel mess management systems rely heavily on manual processes, leading to several operational challenges:

- Students have no visibility into daily menus before visiting the mess.
- No structured mechanism exists for collecting student feedback on meal quality.
- Attendance tracking is done manually using paper registers, prone to errors.
- Complaints are raised informally via WhatsApp groups or verbally, and are often lost.
- Management lacks data driven insights to improve food quality or forecast demand.
- Food wastage occurs due to poor demand estimation and menu planning.

SmartMess addresses all these pain points through a modern, mobile first digital platform.

## 3. Objectives

- Provide real time digital access to daily and weekly mess menus.
- Enable students to rate meals (1 to 5 stars) and submit written feedback.
- Implement a digital attendance marking system for each meal slot.
- Create a structured complaint management portal with status tracking.
- Build an admin dashboard with analytics for data-driven decisions.
- Ensure secure authentication using JWT based token system.
- Deliver a responsive, mobile first user experience.

# 4. System Architecture

SmartMess follows a three tier client server architecture:

## 4.1 Architecture Overview

**Diagram: Three Tier System Architecture**

```
┌──────────────────────────────────────────────────────────────┐
│                          Client Layer                        │
│  ┌──────────────────────────────────────────────────────────┐ │
│  │          React Frontend (Vite + React Router)             │ │
│  │   • Student Dashboard   • Admin Dashboard   • Auth Pages  │ │
│  └──────────────────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────────────────┘
                          │
                          │ HTTPS/REST API
                          ▼
┌──────────────────────────────────────────────────────────────┐
│                       Application Layer                      │
│  ┌──────────────────────────────────────────────────────────┐ │
│  │            Express.js Backend (Node.js)                  │ │
│  │   • Auth Routes   • Menu Routes   • Rating Routes        │ │
│  │   • JWT Middleware   • CORS Handler   • Error Handler    │ │
│  └──────────────────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────────────────┘
                          │
                          │ SQL Queries
                          ▼
┌──────────────────────────────────────────────────────────────┐
│                          Data Layer                          │
│  ┌──────────────────────────────────────────────────────────┐ │
│  │               Supabase PostgreSQL Database               │ │
│  │   • Students   • Admins   • Menus   • Ratings            │ │
│  │   • Attendance   • Complaints   • Notifications          │ │
│  └──────────────────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────────────────┘
```

*Figure 1: Three Tier System Architecture*

## 4.2 Component Breakdown

| Layer | Technology | Responsibility |
|---|---|---|
| Presentation | React 18 + Vite | UI rendering, routing, state management |
| API Gateway | Express.js (Node.js) | REST endpoints, auth middleware, CORS |
| Database | Supabase PostgreSQL | Persistent data storage, SQL queries |
| Authentication | JWT + Bcrypt | Secure login, password hashing |
| Hosting (FE) | Vercel | Static site deployment, CDN |
| Hosting (BE) | Render | Node.js server deployment |

# 5. Database Design

## 5.1 Entity Relationship Diagram (ERD)

### ER Diagram: Entity Relationship Diagram

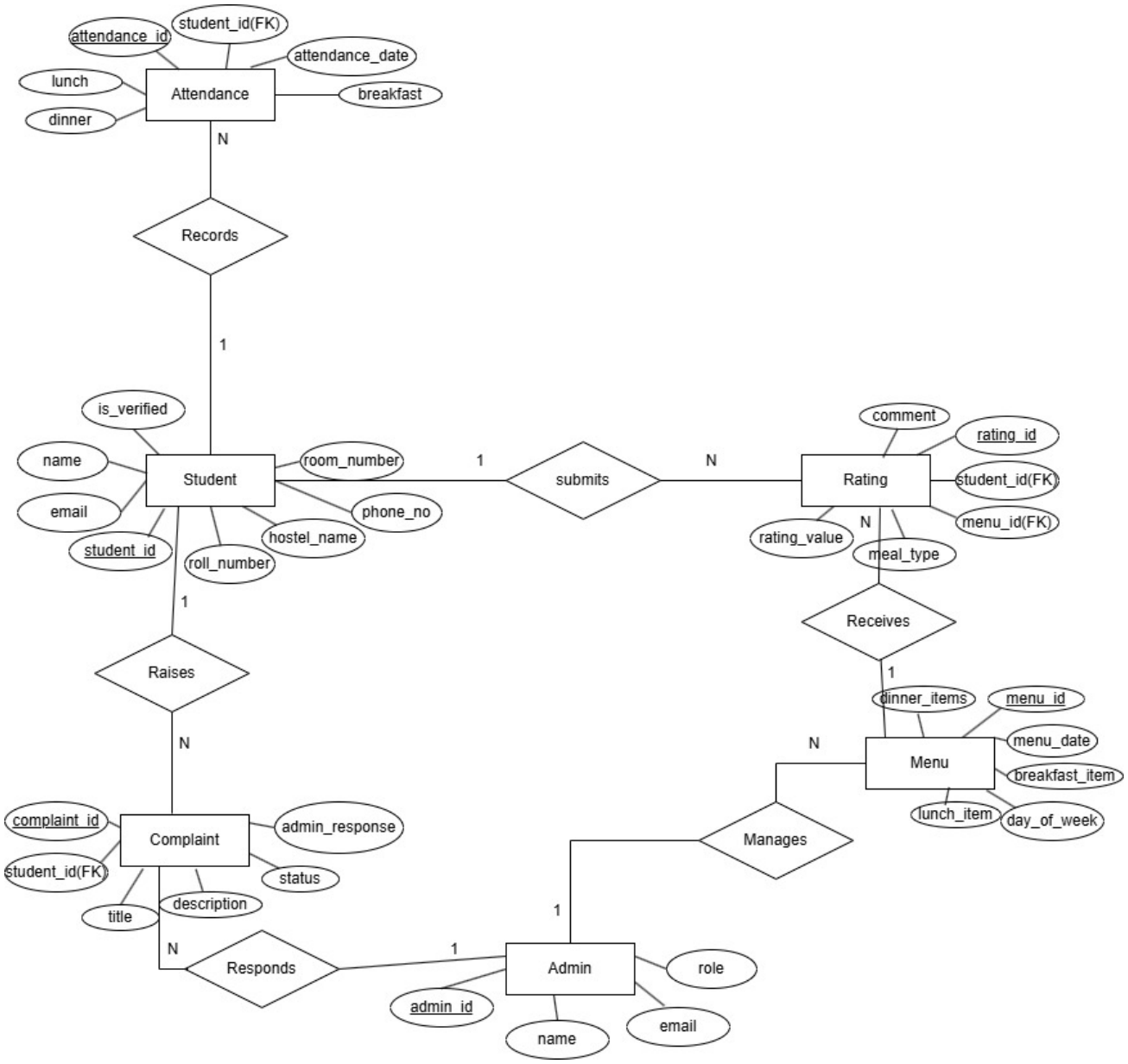


*Figure 2: Entity Relationship Diagram*

## 5.2 Database Schema

| Table | Primary Key | Key Fields | Purpose |
|---|---|---|---|
| students | id (UUID) | name, email, roll_number, hostel_name, is_verified | Student user accounts |
| admins | id (UUID) | name, email, password, role | Admin accounts |
| menus | id (UUID) | date, day, breakfast[], lunch[], snacks[], dinner[] | Daily meal menus |
| ratings | id (UUID) | student_id, menu_id, meal_type, rating, comment | Meal ratings & feedback |
| attendance | id (UUID) | student_id, date, breakfast, lunch, snacks, dinner | Meal attendance records |
| complaints | id (UUID) | student_id, title, description, status, response | Student complaints |

# 6. Technology Stack

## 6.1 Frontend Technologies

| Technology | Version | Purpose |
|---|---|---|
| React | 18.3.1 | UI component library |
| Vite | 5.4.21 | Build tool and dev server |
| React Router | 6.28.0 | Client-side routing |
| Axios | 1.7.9 | HTTP client for API calls |
| React Icons | 5.4.0 | Icon library |
| React Hot Toast | 2.4.1 | Toast notifications |
| Date-fns | 4.1.0 | Date manipulation utility |
| CSS Modules | Built-in | Component-level styling |

## 6.2 Backend Technologies

| Technology | Version | Purpose |
|---|---|---|
| Node.js | 18+ | JavaScript runtime environment |
| Express.js | 4.21.2 | Web framework for REST API |
| Supabase | Latest | PostgreSQL database client |
| JWT | 9.0.2 | Stateless authentication tokens |
| Bcrypt | 5.1.1 | Password hashing and verification |
| CORS | 2.8.5 | Cross-origin request handling |
| Dotenv | 16.4.7 | Environment variable management |

# 7. Features & Modules

## 7.1 Student Module

- Registration with college email, roll number, and hostel details.
- Secure login / logout with JWT token authentication.
- View today's menu (Breakfast, Lunch, Snacks, Dinner).
- View weekly 7 day menu calendar.
- Rate any meal slot with 1 to 5 star rating and optional comment.
- Edit or delete previously submitted ratings.
- Mark digital attendance per meal with a single click.
- Submit complaints with title and detailed description.
- Track complaint status (Pending / In Progress / Resolved).
- Manage personal profile information.

## 7.2 Admin Module

- Secure admin login via dedicated /admin/login route.
- Dashboard with overview statistics (students, ratings, complaints).
- Create and update daily menus for any date.
- Verify or reject new student registrations.
- View and manage all student accounts.
- Browse all meal ratings with filters by date, meal type, and rating.
- Review student complaints and update their resolution status.
- Add official admin responses to complaints.
- Generate attendance reports (meal wise and date wise).

# 8. Screenshots

The following screenshots demonstrate the key screens of the SmartMess application.

## 8.1 Landing Page

### Screenshot: Landing Page — Home Screen

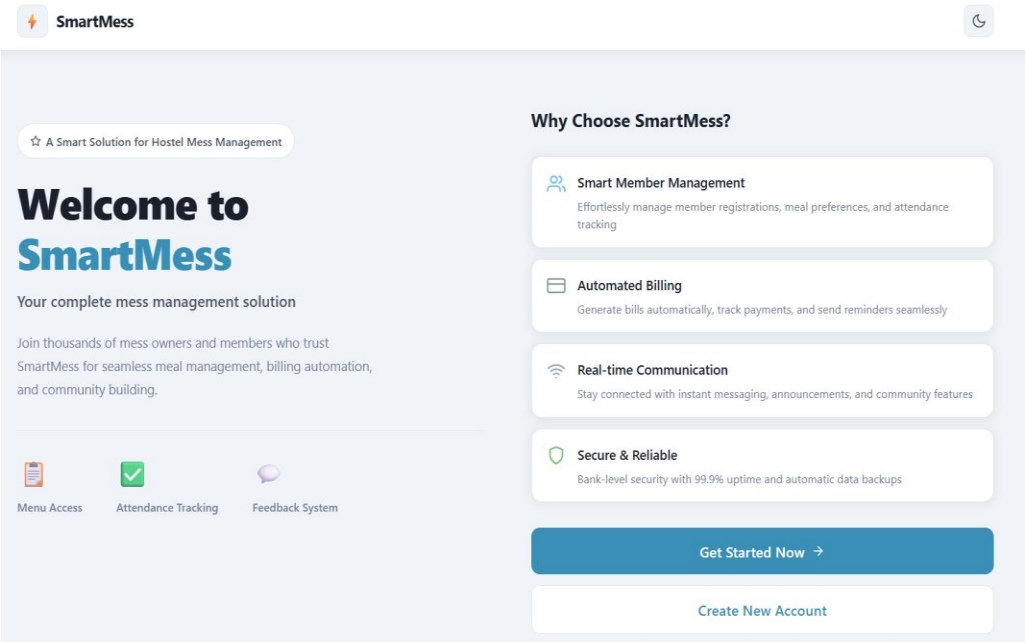


*Figure 3: Landing Page — Home Screen*

## 8.2 Student Registration

### Screenshot: Student Registration Form

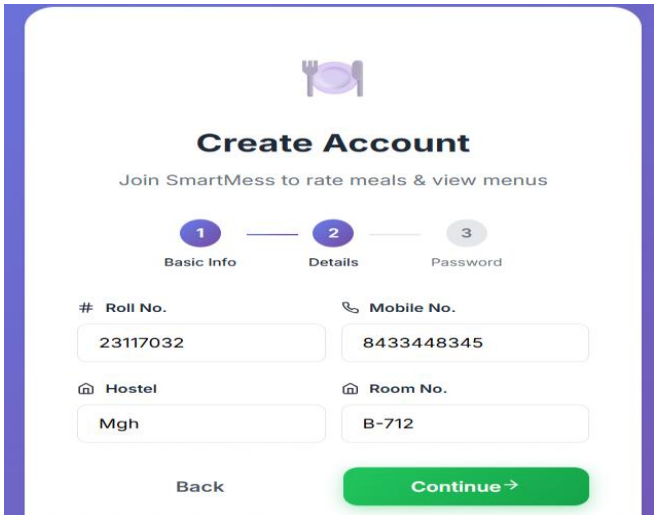


*Figure 4: Student Registration Form*

## 8.3 Student Login

**Screenshot: Student Login Page**

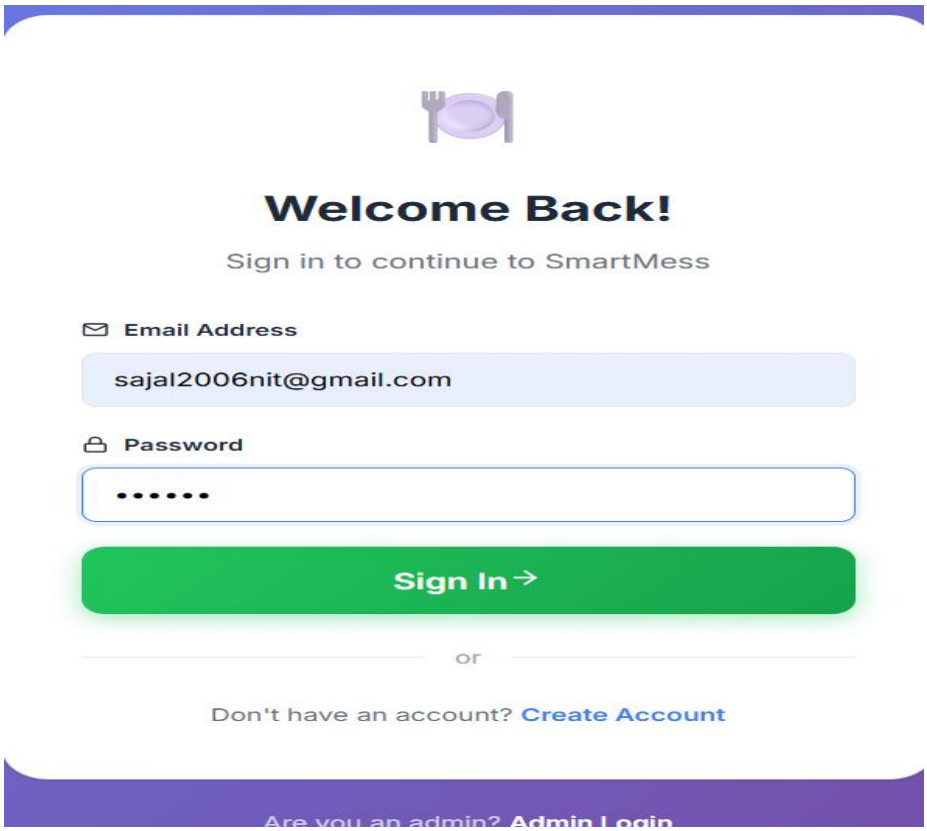


*Figure 5: Student Login Page*

## 8.4 Student Dashboard

### Screenshot: Student Dashboard

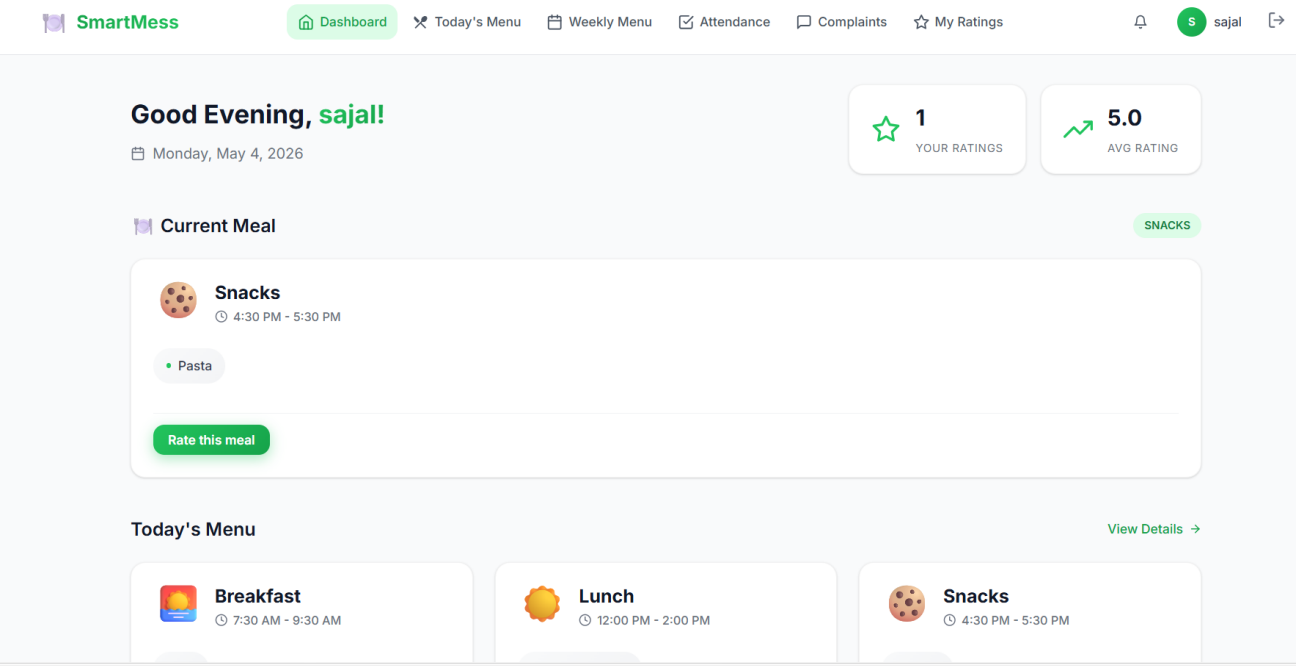


*Figure 6: Student Dashboard*

## 8.5 Today's Menu

### Screenshot: Today's Menu Page

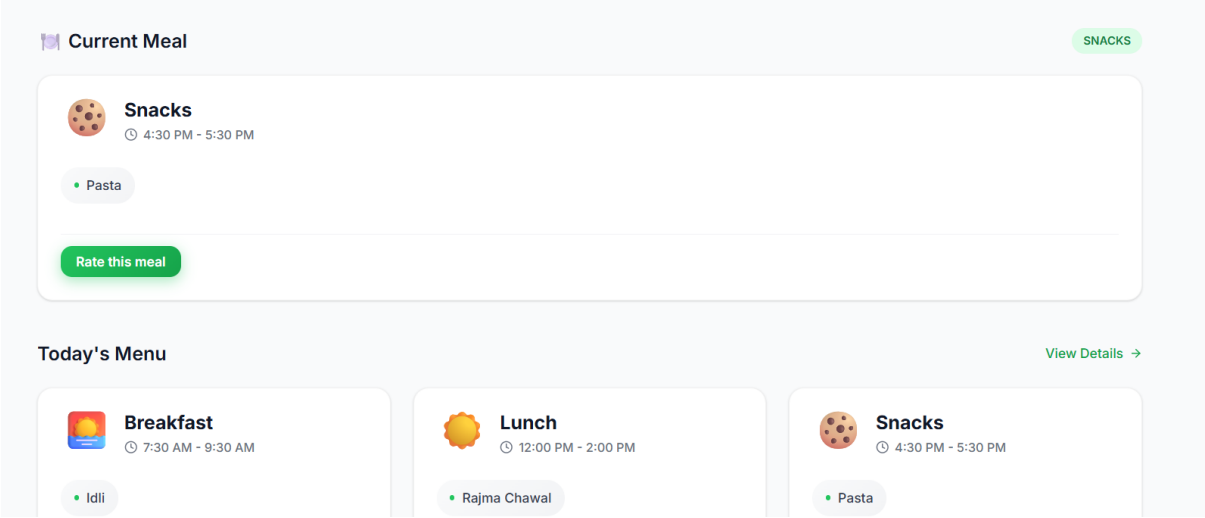


*Figure 7: Today's Menu Page*

## 8.6 Meal Rating

**Screenshot: Meal Rating**

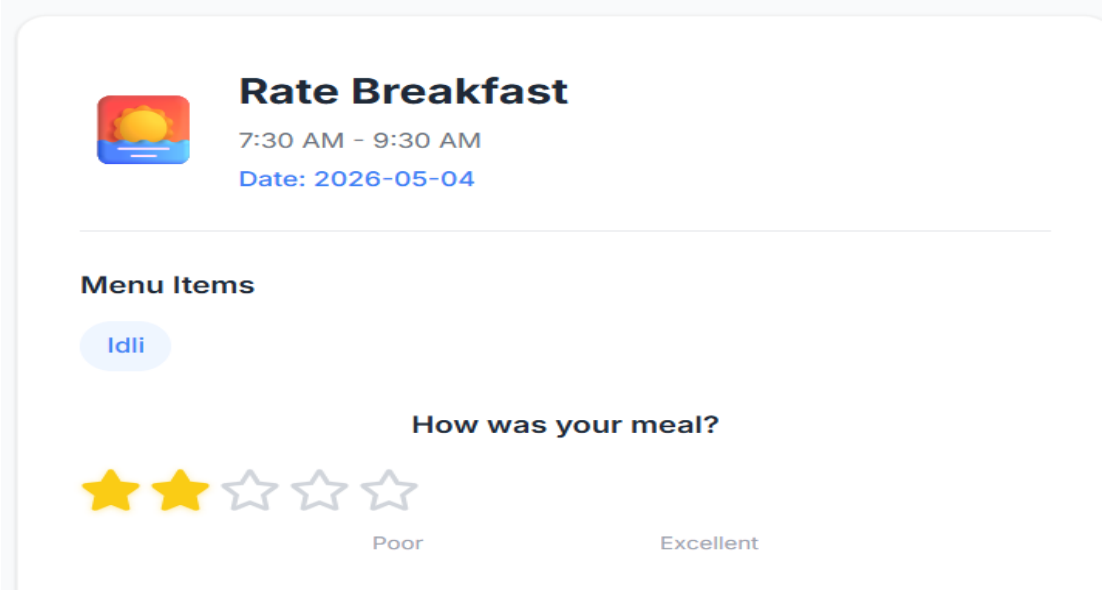


*Figure 8: Meal Rating*

## 8.7 Attendance Marking

**Screenshot: Digital Attendance Page**

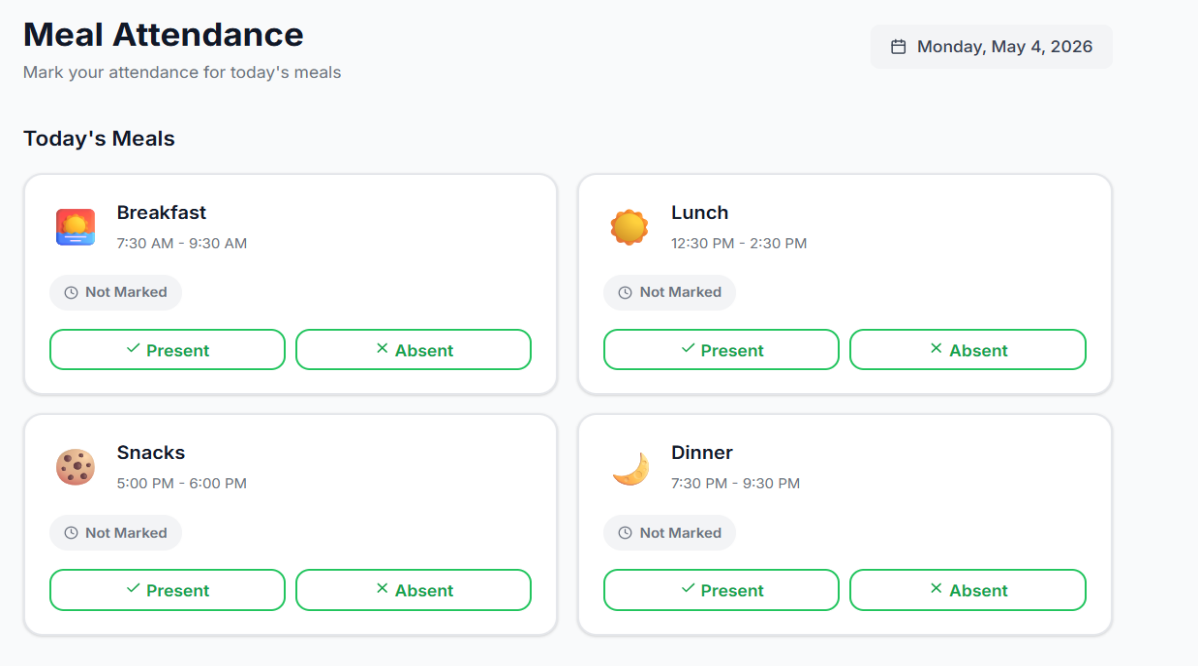


*Figure 9: Digital Attendance Page*

## 8.8 Complaints Portal

**Screenshot: Student Complaints Page**

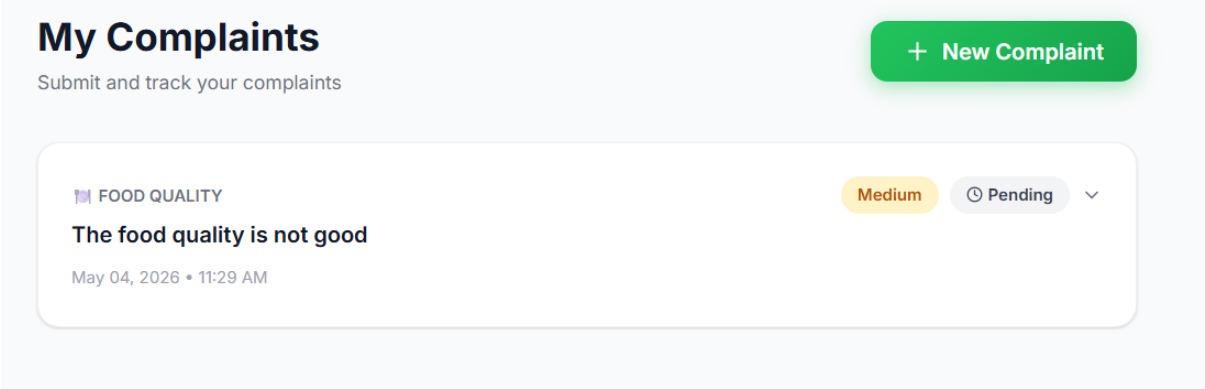


*Figure 10: Student Complaints Page*

## 8.9 Admin Login

**Screenshot: Admin Login Page**

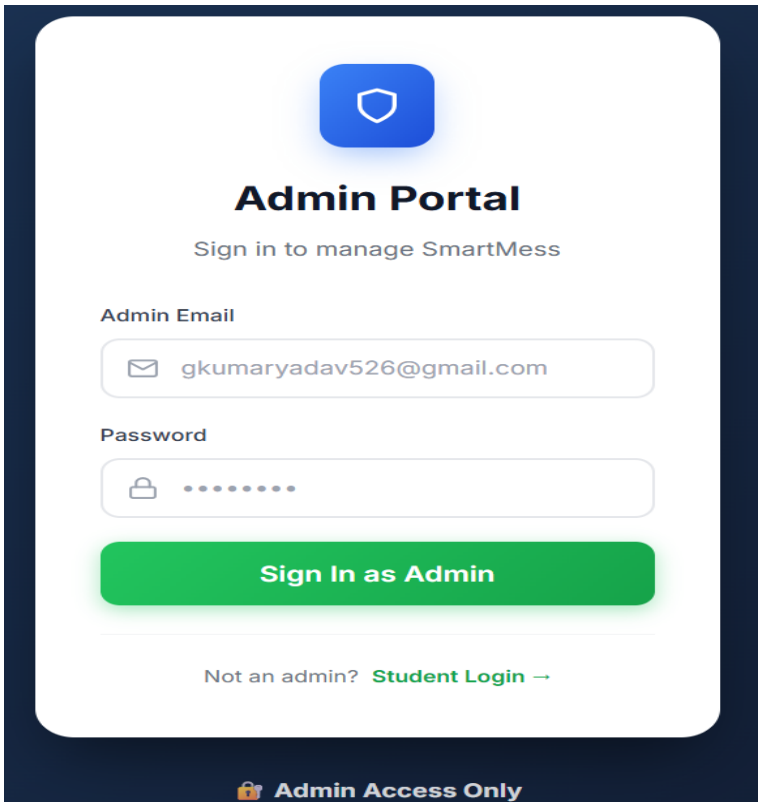


*Figure 11: Admin Login Page*

## 8.10 Admin Dashboard

### Screenshot: Admin Dashboard

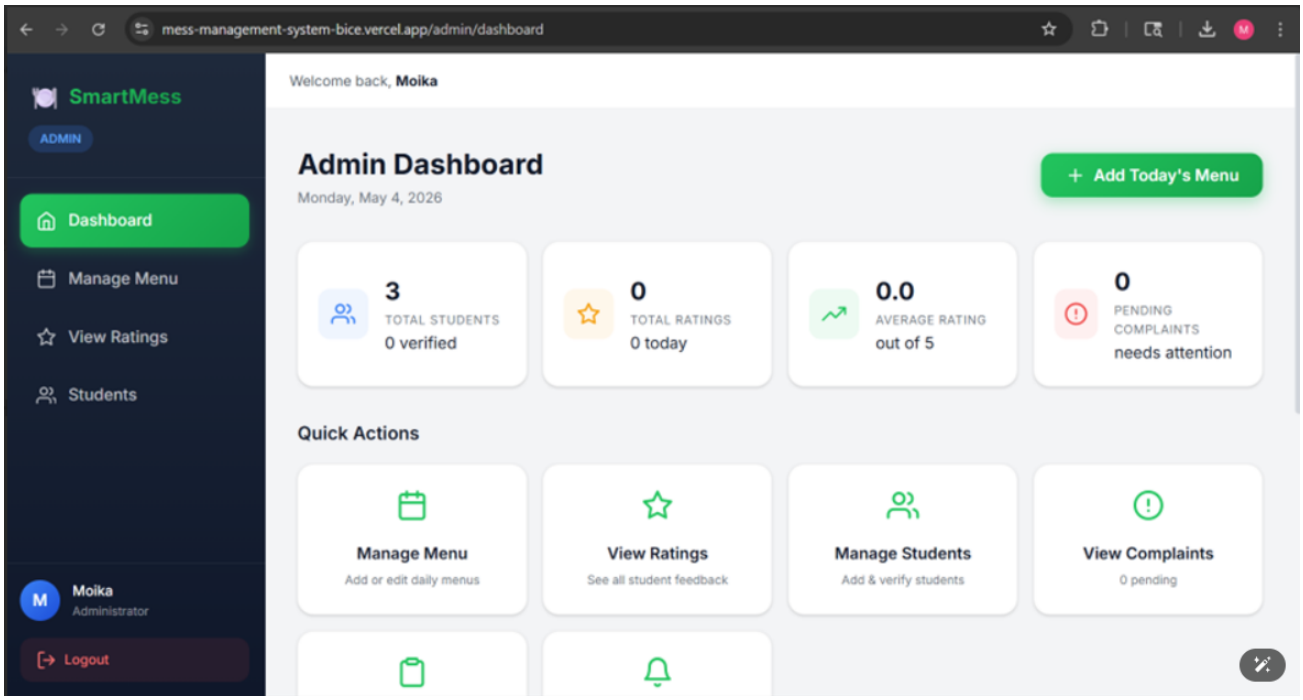


*Figure 12: Admin Dashboard*

## 8.11 Menu Management

### Screenshot: Admin — Menu Management Page

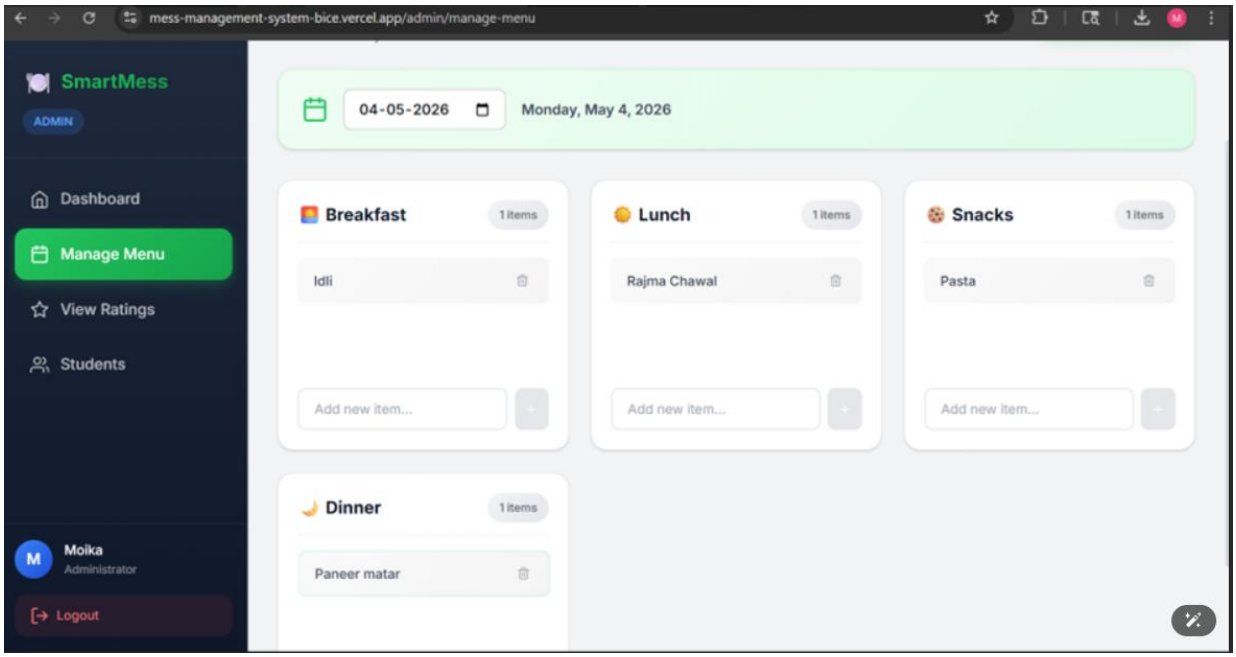


*Figure 13: Admin — Menu Management Page*

## 8.12 Student Verification

**Screenshot: Admin — Manage Students / Verification**

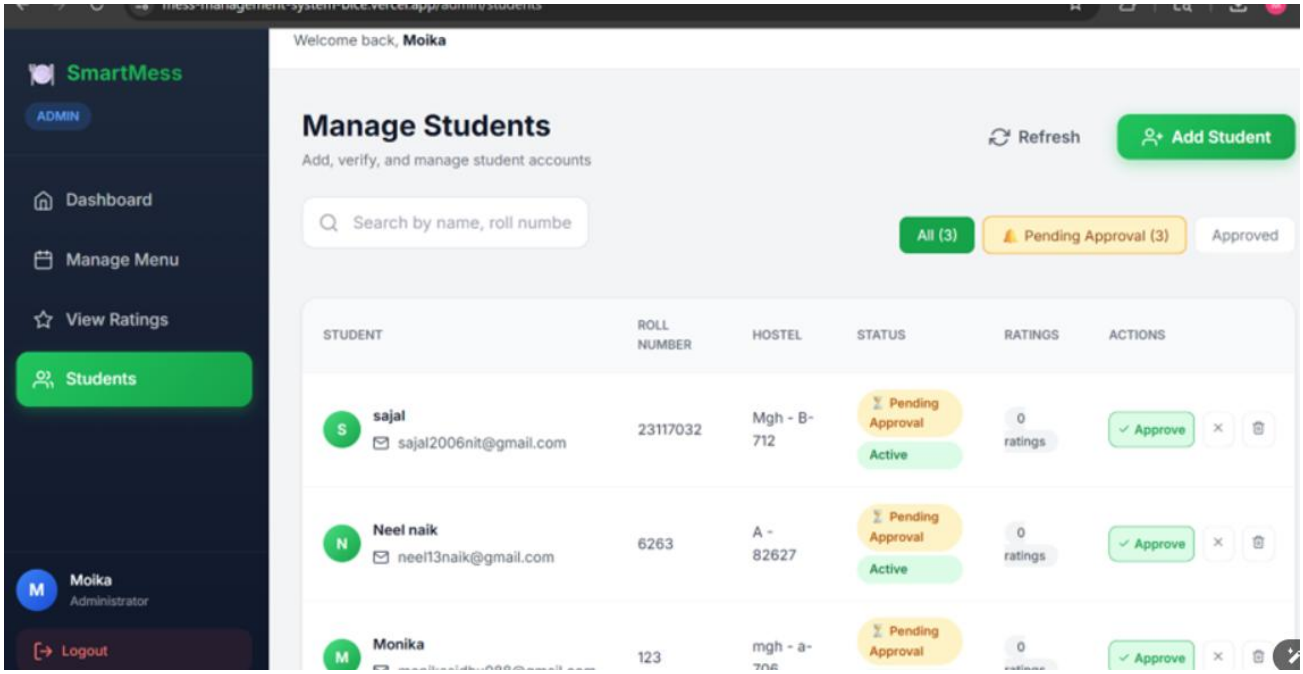


*Figure 14: Admin — Manage Students / Verification*

## 8.13 Complaint Management

**Screenshot: Admin — Complaint Management**

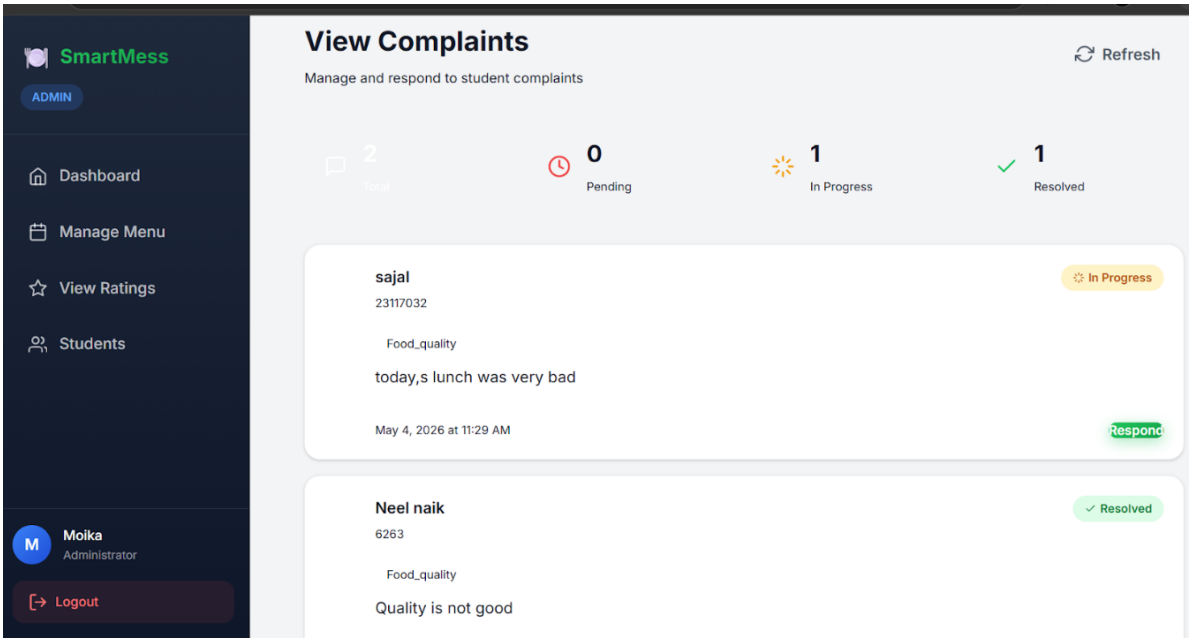


*Figure 15: Admin — Complaint Management*

# 9. Diagrams

## 9.1 Use Case Diagram

**Diagram: Use Case Diagram**

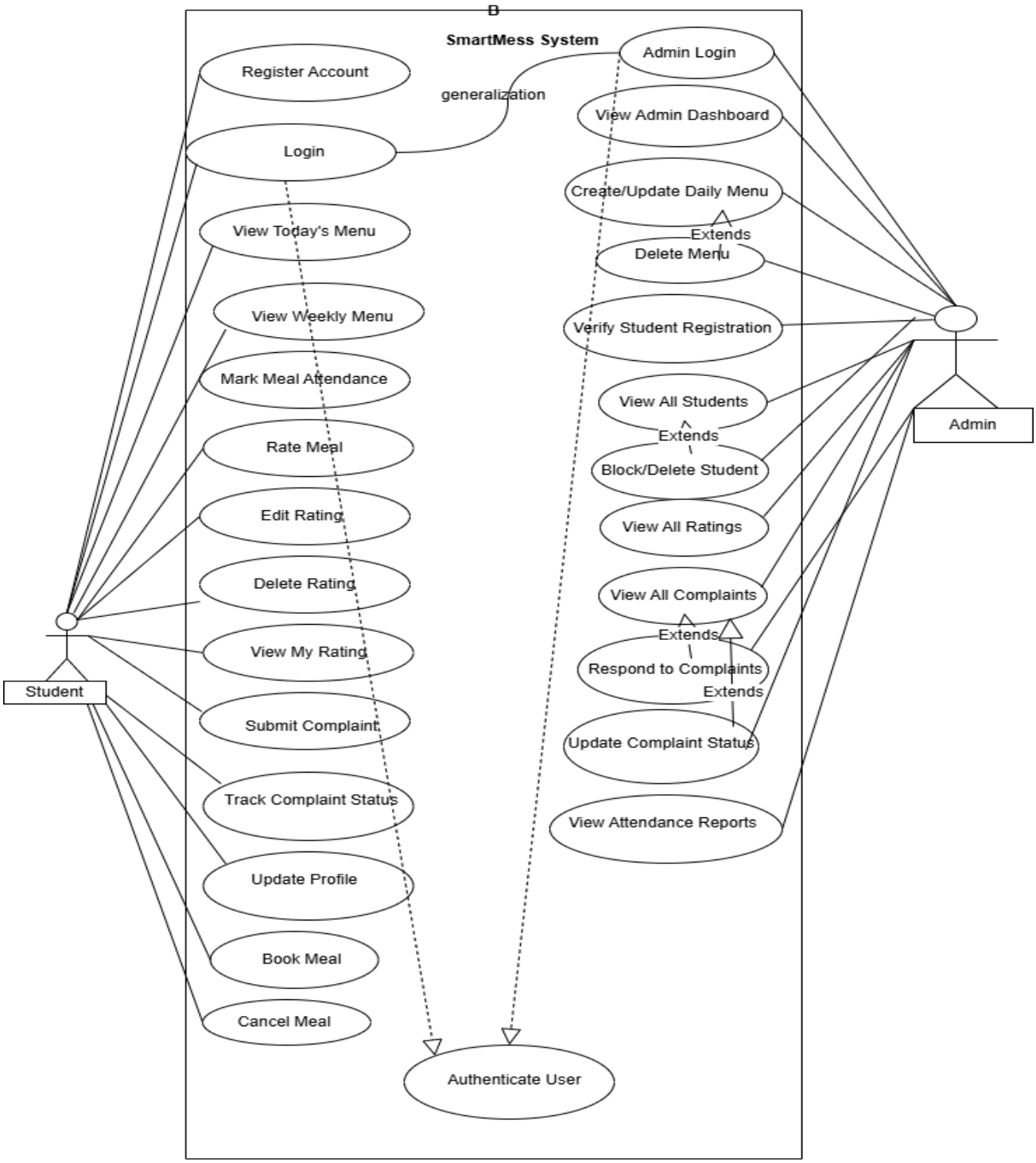


*Figure 16: Use Case Diagram*

## 9.2 Data Flow Diagram (Level 0 )

**Diagram: DFD Level 0**

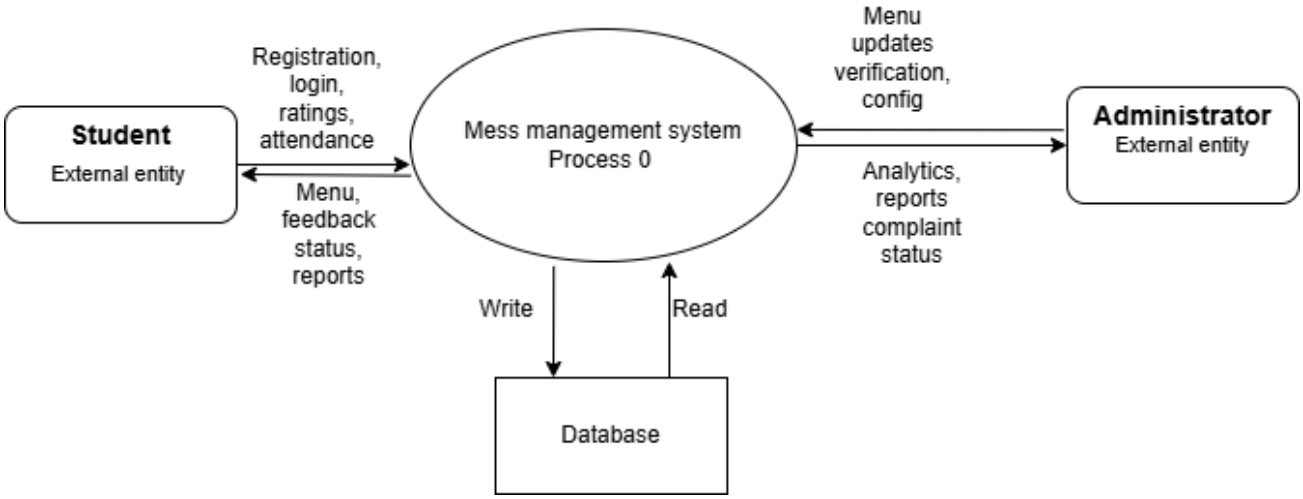


*Figure 17: DFD Level 0*

## 9.3 Data Flow Diagram (Level 1)

**Diagram: DFD Level 1**

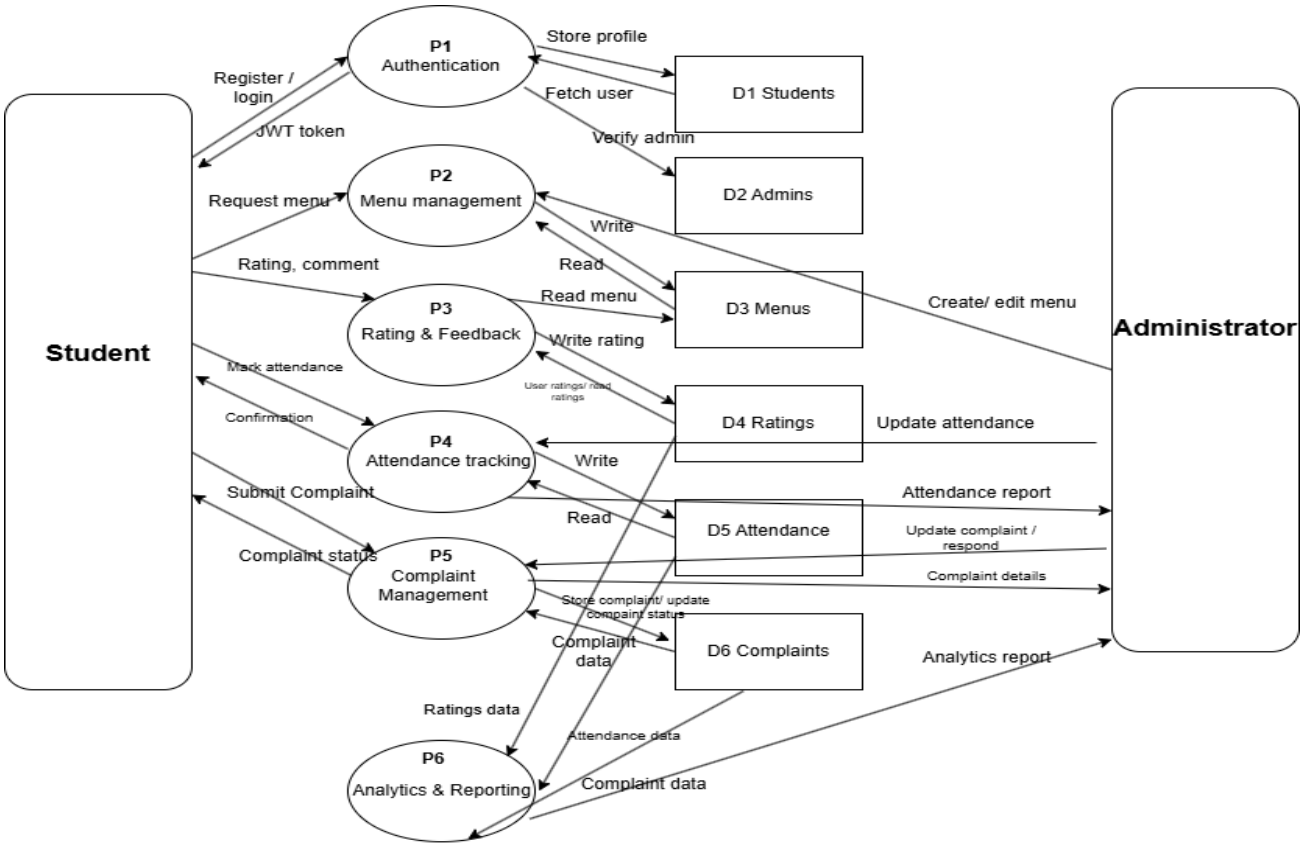


*Figure 18: DFD Level 1*

## 9.4 Project Folder Structure

**Diagram: Project Directory Structure Tree**

```
SmartMess/
├── frontend/                # React Frontend
│   ├── src/
│   │   ├── components/        # Reusable UI Components
│   │   │   ├── layout/        # Layout components (Navbar, Footer)
│   │   │   └── ui/            # UI components (Button, Card, Modal)
│   │   ├── pages/             # Page components
│   │   │   ├── admin/         # Admin pages
│   │   │   ├── Landing.jsx     # Landing page
│   │   │   ├── Dashboard.jsx   # Student dashboard
│   │   │   └── ...
│   │   ├── context/           # React Context (Auth)
│   │   ├── hooks/             # Custom React hooks
│   │   ├── services/          # API service layer
│   │   ├── styles/            # Global styles
│   │   └── utils/             # Utility functions
│   ├── public/                # Static assets
│   ├── package.json           # Frontend dependencies
│   └── vite.config.js         # Vite configuration
│
├── backend/                 # Express Backend
│   ├── src/
│   │   ├── config/            # Configuration files
│   │   │   └── database.js     # Supabase client
│   │   ├── controllers/       # Route controllers
│   │   │   ├── adminController.js
│   │   │   └── studentController.js
│   │   ├── models/            # Database models
```

```
│   │   │   ├── Student.js
│   │   │   ├── Menu.js
│   │   │   └── Rating.js
│   │   ├── routes/            # API routes
│   │   │   ├── authRoutes.js
│   │   │   ├── adminRoutes.js
│   │   │   └── studentRoutes.js
│   │   ├── scripts/           # Utility scripts
│   │   │   └── createAdmin.js  # Admin creation script
│   │   └── server.js          # Express server entry
│   ├── .env                   # Environment variables
│   ├── package.json           # Backend dependencies
│   └── database_schema.sql    # Database schema
│
├── .gitignore                 # Git ignore file
├── DEPLOYMENT.md              # Deployment guide
└── README.md                  # This file
```

# 10. API Documentation https://messmanagementsystem-4ngq.onrender.com

Base URL (Development): http://localhost:5000/api

Base URL (Production): https://messmanagementsystem-4ngq.onrender.com

All protected routes require: Authorization: Bearer <JWT_TOKEN> in request header.

## 10.1 Authentication Routes

| Method | Endpoint | Description | Auth |
|---|---|---|---|
| POST | /auth/register | Student registration | No |
| POST | /auth/login | Student login | No |
| POST | /auth/logout | Student logout | Yes |
| POST | /admin/login | Admin login | No |
| GET | /auth/me | Get current user info | Yes |

## 10.2 Menu Routes

| Method | Endpoint | Description | Auth |
|---|---|---|---|
| GET | /menus/today | Get today's menu | Student |
| GET | /menus/weekly | Get weekly menu | Student |
| GET | /menus/date/:date | Get menu by specific date | Student |
| POST | /admin/menus | Create or update menu | Admin |
| DELETE | /admin/menus/:id | Delete a menu entry | Admin |

## 10.3 Rating & Complaint Routes

| Method | Endpoint | Description | Auth |
|---|---|---|---|
| GET | /ratings/my-ratings | Get student's own ratings | Student |
| POST | /ratings | Submit a new rating | Student |
| PUT | /ratings/:id | Update existing rating | Student |
| DELETE | /ratings/:id | Delete a rating | Student |
| GET | /admin/ratings | View all ratings | Admin |
| GET | /complaints | Get student's complaints | Student |
| POST | /complaints | Submit a complaint | Student |
| GET | /admin/complaints | View all complaints | Admin |
| PUT | /admin/complaints/:id | Update complaint status | Admin |

# 12. Deployment

| | |
|---|---|
| **Frontend URL** | https://mess-management-system-bice.vercel.app |
| **Backend URL** | https://messmanagementsystem-4ngq.onrender.com |
| **Database** | Supabase Cloud (PostgreSQL) |
| **Frontend Host** | Vercel — Auto-deploy from GitHub main branch |
| **Backend Host** | Render — Node.js web service |

The application is fully deployed and accessible online. The frontend is hosted on Vercel with automatic HTTPS and CDN. The backend runs on Render's Node.js environment with environment variables securely configured. The Supabase database is cloud hosted with automatic backups.

## 13. Future Scope

- Mobile application development using React Native for iOS and Android.
- WhatsApp and email notification integration for menu updates and approvals.
- Payment integration for digital mess bill generation and collection.
- Nutritional information display for each meal item.
- Allergen warnings and dietary preference filters.
- QR code-based attendance marking at mess entry points.
- AI-powered menu suggestions based on historical ratings.
- Export reports to PDF or Excel for administrative records.
- Multi-language support (Hindi, regional languages).
- Push notifications for real time menu updates.

## 14. Conclusion

SmartMess successfully addresses the key challenges in hostel mess management by providing a comprehensive digital platform. The application bridges the communication gap between students and mess administration through real time menu visibility, structured feedback collection, digital attendance tracking, and complaint management.

Built on the MERN stack with Supabase PostgreSQL as the database, the system demonstrates modern full stack development practices including JWT based authentication, RESTful API design, component based UI architecture, and cloud deployment. The application is live at smartmesslms.vercel.app and serves as a complete solution ready for real world hostel deployment.

The project highlights the practical application of web technologies to solve everyday campus problems and demonstrates proficiency in both frontend and backend development.

## 15. References


- React Documentation — https://react.dev
- Express.js Documentation — https://expressjs.com
- Supabase Documentation — https://supabase.com/docs
- JSON Web Tokens — https://jwt.io
- Vercel Deployment — https://vercel.com/docs
- Render Deployment — https://render.com/docs
- GitHub Repository — https://github.com/monikaasidhu/Mess-Management-System
- Live Application — https://mess-management-system-bice.vercel.app